MonthlyMuse

Presentation script — two speakers

AI-Powered Automated Message Generator for Monthly Posting · ~14 minutes + Q&A · **Speaker A** (blue shirt) opens and handles architecture / engineering · **Speaker B** (green shirt) handles the AI maths and runs the live demo · status as of November 2026

14 slides + 2 appendices | 14 minutes | live demo built in | alternating speakers | hosted LLM, template fallback | Q&A prep included

How to use this script

- Text inside a coloured box is **verbatim** — say it as written or in your own words; the meaning must stay.
- SPEAKER A and SPEAKER B mark who owns the slide.
- **Handoff lines** (in yellow) are spoken by the named speaker to pass the floor — say them out loud so the audience knows who is talking next.
- Timings are cumulative from the start. Keep to them; if you run long, trim the roadmap slide, never the demo.
- Run the demo on the deployed site (`https://monthly-muse-bay.vercel.app`) or, if the network is unreliable, on `http://localhost:3000` with `demo@monthlymuse.app` / `monthlymuse-demo`. The December 2026 candidate batch is already generated, so the demo starts one click from the review screen.
- Two appendices (pipeline stage reference, algorithm reference) are backup material for questions — you will only need them if the audience pushes past slide 7.

> **What is different from last term's script:** candidates are now written by a **hosted language model** (Groq, `openai/gpt-oss-120b`) through a provider chain, not by the template engine. The template engine still exists and is still used — but only as an explicit, visible fallback, and only when `MM_ALLOW_TEMPLATE_FALLBACK=true` . Every batch records which provider wrote it, so the review screen can label it. If a model call genuinely fails the request is marked *failed* and the previous batch is kept; nothing is silently substituted. Say this plainly if someone asks — it is one of the project's better design decisions.

Slide 1 — Title `0:00 - 0:30`

MonthlyMuse — AI-Powered Automated Message Generator for Monthly Posting

Opening 0:30

SPEAKER A

> "Good morning everyone. This is MonthlyMuse — a web application that prepares your monthly social post *before you even ask for it*. You define a monthly plan once, and every month the system retrieves your own history, picks writing styles, asks a hosted language model for six candidate messages, then ranks and diversifies them with our own maths engine and hands you the best three with a transparent explanation of why they ranked where they did."

SPEAKER B

> "Everything you'll see today is built, deployed and running — a hundred and sixty-six automated tests pass, the API is serving live traffic, and we'll be demoing the real application, not slides of it."

> **Handoff — A:** "To understand why this is worth building, let's start with the problem."

Slide 2 — The problem `0:30 - 1:30`

Small businesses post monthly — and struggle every time

Context 1:30

SPEAKER A

> "Think about a small café, a salon, or a tutor. They know they should post once a month. But on the day, they face a blank box: what do I write, does it sound like me, have I already said this last month? The result is either no post, or a generic one that doesn't sound like their brand."

> "Existing AI writers give you one shot and no explanation. You get a paragraph, you don't know why it was chosen, and next month the tool has forgotten everything you liked. Nothing compounds."

> **Handoff — A:** "So our goal was specific. B will explain it."

Slide 3 — The goal `1:30 - 2:15`

Goal: prepare, don't react — and prove it improves

Objective 2:15

SPEAKER B

> "Our goal in one sentence: the system should prepare the monthly post *before* the user asks, show three genuinely different candidates with a visible score breakdown, and then measurably improve from every choice the user makes."

> "Three promises. **Automatic** — a scheduler, not a reminder. **Transparent** — you can see every feature that produced the score, and which model wrote the text. And **adaptive** — your selection changes next month's output."

> **Handoff — B:** "Here's what that looks like. A takes you through the flow."

Slide 4 — How it works (user flow) `2:15 - 3:15`

The monthly loop

Product flow 3:15

SPEAKER A

> "The loop is short. First, you create a **Monthly Plan** — topic, audience, platform, tone, posting day. Then the scheduler detects the upcoming month, checks whether a calendar occasion actually relates to your objective, and runs the pipeline automatically."

> "You get a notification: three ranked candidates are waiting. You open the review screen, pick one, edit one, or reject one with a reason. You approve it, it lands in the calendar, and it's marked published."

> "If none of the three fits, **Regenerate all** runs the whole pipeline again from the *saved brief* — the plan details are read back from the request, never re-inferred from the candidate text. Every action — select, edit, like, dislike, reject-with-reason — is recorded as a feedback event. That's where the learning starts."

> **Handoff — A:** "But behind that simple loop sits a fairly serious engine. B."

Slide 5 — Architecture `3:15 - 4:30`

System architecture — three pieces, one database, deployed

Engineering 4:30

SPEAKER B

> "The system is three pieces. A **Next.js 14 frontend** on Vercel that only ever talks to the public API — no secrets in the browser. A **FastAPI backend** on Render with **fifty-eight route handlers** across forty-five paths, covering auth, plans, cycles, messages, analytics, calendar and admin health."

> "And a **scheduler worker** on Render, running on APScheduler, sharing the same code and database but started with a different command. The API is stateless, so it scales by adding instances."

> "Underneath is a **sixteen-table schema**: users, preferences, audiences, topics, occasions, plans, cycles, generation requests, messages, feedback events, tone statistics, scheduled posts, metrics, notifications, job runs and a daily analytics rollup. SQLite by default; one environment variable makes it Postgres, which is what the deployed stack can switch to without code changes."

> **Handoff — B:** "The interesting part is what runs inside that backend. A."

Slide 6 — The AI pipeline `4:30 - 6:00`

Fourteen stages, one language-model call

Core system 6:00

SPEAKER A

> "Generation is fourteen numbered stages. It starts by **validating the brief** — month, topic, audience, platform, tones, length, banned words — then **assembles context**, including the occasion only if two gates pass. Then **preprocessing**: keyword extraction and text normalisation."

> "Stage four builds a 384-dimensional **intent vector** from the brief. Stage five is **retrieval**: cosine similarity against your whole history matrix picks three style exemplars and six recent posts, so the model sees how you actually sound. Stage six picks three **style arms** — a tone and a structure per candidate — with Thompson sampling, two exploiting what you liked and one exploring."

> "Stage seven builds the prompt, and stage eight makes **one call** for **six candidates**. One call, six candidates: that's deliberate — it's the cost-efficient way to have something to choose from, and it keeps all six candidates comparable because they share a prompt."

> "Then the filter chain: hard constraints on length, banned words and emoji level, Jaccard deduplication of opening lines, and embeddings. Features, scores, MMR for stylistic diversity, Monte-Carlo confidence, and finally the explanation payload — which is what you actually see on screen."

> **Handoff — A:** "Now the maths. B."

Slide 7 — The maths 6:00 – 7:30

Scoring: s = F · w, then MMR, then confidence

Mathematical core 7:30

SPEAKER B

"Each candidate becomes a row of **six features**, each normalised to zero to one: **relevance** — cosine between the candidate and the intent vector; **tone** — probability from our softmax classifier; **personalisation** — dot product with your preference vector; **novelty** — one minus the maximum cosine against your history; **length** — a z-score against what you usually write; and **history** — a Beta estimate of how this style did before."

"The score is a dot product, **s equals F times w**, with default weights of point three, point two, point two, point one five, point one and point oh five. Those weights are not fixed: when you reject something for being *too similar*, the novelty weight steps up. The update is a **constrained gradient step** — learning rate point oh one, step size point oh oh five, bound eight — projected back onto the simplex so the six weights always sum to one and never go negative."

"The score alone would give you three near-identical posts. So we add **MMR** — maximal marginal relevance — at lambda zero point seven, which trades relevance against diversity, with a quota that prefers a style arm not yet represented. That's why candidate three is tagged *'exploring a new style'*."

"Confidence comes from a **Dirichlet Monte-Carlo simulation**: two hundred draws from a Dirichlet distribution centred on your weights, giving the probability that each candidate is rank one. It's seeded at forty-two, so the same inputs always give the same answer — that determinism is what makes golden-value testing possible. For rates in analytics we use **Wilson intervals**, because a naive percentage lies when n is small."

Handoff — B: "Rather than describe the output, let's show it. A drives the demo."

Slide 8 — LIVE DEMO: review screen `7:30 - 9:30`

Demo: three ranked candidates with score breakdowns

Live — deployed site or localhost:3000 9:30

SPEAKER A

"This is the live app. The dashboard says '*December post ready for review — three candidates*'. That message came from the scheduler, not from a button press."

Click → Review candidates.

"First, provenance, because it's the honest answer to 'what made this?': **Provider: Groq**, **Model: openai/gpt-oss-120b**, **Source: monthly schedule**, and when it ran. If a batch ever came from the offline template engine instead, this line would say so and an amber banner would explain why — the UI never labels template text as model output."

"Below that is the brief the batch was actually built from — topic, audience, platform, tones, scheduled date, keywords. Read from the database, not guessed from the candidates. Then the three cards: a **score ring**, a **confidence band**, word and character counts, and how similar it is to your history — twenty percent, thirty-five, fifty-three. All three marked *Fresh*, so we're not repeating ourselves."

Click → Score breakdown on candidate 1.

"This is the transparency we promised. Six bars: relevance seventeen, tone thirteen, personalisation sixty-one, novelty eighty, length eighty-five, history sixty-seven. You can see exactly which feature earned the top rank — and which one held it back."

"And this list is the **why-ranked** explanation in plain English — '*fits your usual voice at sixty-one percent*' and '*twenty percent different from your nearest past post*' — with a link to the exact post it's being compared against."

Optional if time allows: click Compare, then Embedding map; then Regenerate all and let the provider line update.

Handoff — A: "Now watch what happens when we give feedback. B?"

Slide 9 — LIVE DEMO: the learning loop `9:30 - 10:45`

Demo: one click changes the next month

Live — feedback → personalization 10:45

SPEAKER B

> "I'll like this first style. Watch the network — the feedback call returns two hundred OK, and the request refetches."

> **Click → 'Like this style', then point at the Recent activity feed on the dashboard.**

> "That single click updated four things at once. It moved the **preference vector** with an exponential moving average toward that candidate's embedding. It updated the **Beta statistics** for that style arm — the alpha went up — so Thompson sampling will favour it next time. It nudged the **weight vector** with that constrained gradient step. And it updated your **tone counts**."

> "Reject does the same thing with a reason attached — if you reject for *'too similar'*, the novelty weight goes up; *'wrong tone'* pushes tone. That's the difference between a generator and a system that learns."

> **Handoff — B:** "A shows where that shows up over time."

Slide 10 — Analytics `10:45 - 11:45`

Analytics: proving the improvement

Evaluation 11:45

SPEAKER A

> "Three dashboards. **Usage** — how many messages generated, time to approve. **Content** — tone mix, topic distribution, novelty trend across months, entropy of the topic distribution, and a repetition alert that fires when two posts are more than eighty-five percent similar."

> "And **Quality** — selection rate with a Wilson confidence interval, top-one agreement between the rank-one candidate and what you actually chose, mean reciprocal rank, and the correlation between score and choice. There's also a chi-square panel testing whether tone and selection are independent."

> "Right now: nine messages generated, twenty-two percent selection rate with a ninety-five percent interval of six to fifty-five percent. The interval is wide because n is nine — which is exactly why we show it, instead of pretending a small sample is a result."

> **Handoff — A:** "B takes the technology summary."

Slide 11 — Technology stack `11:45 – 12:45`

Technology stack

Engineering choices 12:45

SPEAKER B

> "**FastAPI** on Python 3.10 with Pydantic 2 for validation and **SQLAlchemy 2** over SQLite by default — one environment variable and it's PostgreSQL, which is the deployment path we keep open."

> "Generation goes through an **adapter layer**: Groq, Gemini, OpenAI, Anthropic and Ollama behind one interface, each with its own **circuit breaker**, retry with backoff, tolerant JSON parsing and cost metering. The chain tries the primary provider, then any other configured hosted provider. The **template engine** is the last resort, and only if the deployment explicitly opts in — otherwise the request fails visibly rather than faking success."

> "The ranking engine is **pure NumPy and SciPy** — no GPU, no ML framework. Our own 384-dimensional embedder (sentence-transformer MiniLM in development, deterministic hashing in the deployed build), our own preprocessing, our own softmax logistic-regression tone classifier trained on eight hundred labelled rows. **Exa** web search can add live context to an occasion when a key is configured."

> "On the front, **Next.js 14** with TypeScript, TanStack Query, React Hook Form with Zod, Recharts for the visualisations, Tailwind on our own design tokens. Deployment is **Vercel for the frontend**, **Render for the API and the worker**, with **GitHub Actions** running the test suite on every push."

> **Handoff — B:** "A on quality."

Slide 12 — Quality & testing `12:45 - 13:30`

Quality: 166 tests, including every golden value

Verification 13:30

SPEAKER A

> "A hundred and sixty-six tests, all passing. Thirty-five are **golden-value tests** — we wrote the expected number down first, then made the code produce it. The gradient step lands on exactly 0.161, the Wilson interval for three successes in ten trials is 0.108 to 0.603, and MMR picks candidates A, C and F."

> "The rest is behavioural: twenty-nine pipeline-behaviour tests, twenty-two occasion-gating tests, nineteen date-and-cycle tests, twelve text-quality tests, a full API journey, six regeneration tests, six webhook tests, six demo-account tests, four scheduler tests and three for the health endpoint."

> "Type-checking and the production build are clean, CI runs pytest on Python 3.10 and 3.11 plus the frontend build on Node 20, and we verify deployment from outside: the health endpoint reports the **git commit** it is running, so we can prove which build is live instead of assuming."

> **Handoff — A:** "B finishes."

Slide 13 — Live now & what's next `13:30 - 14:00`

Where it stands, and what's next

Closing 14:00

SPEAKER B

> "The MVP is **complete** — every row of the scope is built, tested and running. The code is on GitHub, there's Docker Compose for the full stack, a CI pipeline, and the application is **deployed**: frontend on Vercel, API and scheduler worker on Render, with the model provider configured in production so candidates are AI-written rather than template-written."

> "Next: managed Postgres for durable data on the free tiers, a Playwright end-to-end test in CI, smarter fallback telemetry so we notice provider degradation before a user does, and the Version 2 items — platform publishing, engagement import, teams."

SPEAKER A

> "To leave you with the one idea: this isn't a text box with an API behind it. It's a system that prepares, explains itself, and gets measurably better every month you use it."

> **"Thank you — we're happy to take questions."**

Appendix A — Pipeline stage reference

The fourteen stages, exactly as implemented (`app/ai/pipeline.py`)

#	Stage	What it does · key parameters
1	Validate	Normalises the brief, collects warnings; refuses malformed input before spending a model call
2	Context	Season and occasion resolved by the caller through the relevance + date gates; keyword extraction from topic, occasion and fact
3	Preprocess	Normalise, tokenise, lemma, coverage keywords for the hard constraints
4	Intent vector	384-d embedding of the brief text (topic, audience, purpose, tones, CTA, keywords)
5	Retrieval (RAG)	Cosine over the history matrix H: three style exemplars + six most recent posts
6	Style arms	Thompson sampling over tone × structure arms, Beta posteriors, 2 exploit + 1 explore
7	Prompt	System + user prompt carrying brief, arms, exemplars, recent posts, voice notes, banned words
8	Generate	**One** provider call for six candidates; provider chain, breaker, retry; template only if opted in
9	Filter	Hard constraints: length per platform, banned words, emoji level, CTA, keyword coverage, near-duplicate history
10	Embed	384-d L2-normalised candidate embeddings; similarity matrices $E \cdot E^T$ for dedupe and MMR
11	Features	Six-feature row F: relevance, tone, personalisation, novelty, length, history
12	Score	$s = F \cdot w$ with the user's current weight vector
13	Rank (MMR)	$\lambda = 0.7$ with style-arm quotas so the top three differ; trace records the order and the top-3-by-score it replaced
14	Confidence + explain	200-draw Dirichlet Monte-Carlo → P(rank 1) and band; plain-English reasons; PCA coordinates for the embedding map

> **Failure path:** if the provider returns nothing usable and the deployment has not opted into templates, the request is marked `FAILED` with the reason stored on the request — the previous batch stays intact and the UI says so. No silent substitution anywhere in the pipeline.

Appendix B — Algorithm reference (numbers you may be asked for)

Every formula, its parameters, and why it is there

Algorithm	Definition	Parameters / notes
Embedding	384-d L2-normalised vector per text	MiniLM (`sentence-transformers`) in dev; 384-d feature hashing in the deployed build; stored as float32 BLOB
Cosine similarity	$\langle a,b\rangle / (\|\|a\|\|\|\|b\|\|)$	History retrieval, novelty, duplicate detection, embedding map
Feature vector	F = [relevance, tone, personalisation, novelty, length, history] $\in [0,1]^6$	History feature is a Beta posterior mean; length is a clipped z-score against learned stats
Score	$s = F \cdot w$	default w = (.30, .20, .20, .15, .10, .05)
Online weight update	$w \leftarrow \text{proj}_{\text{simplex}}(w + \eta \cdot \text{reward} \cdot \nabla)$	η = 0.01, step bound β = 8, λ = 0.005; weights stay non-negative and sum to 1
Preference vector	$u \leftarrow (1-\alpha)\cdot u + \alpha \cdot e_{\text{chosen}}$	EMA over chosen/copied candidate embeddings; drives personalisation = $\langle e, u\rangle$
Thompson sampling	draw $\theta \sim \text{Beta}(\alpha, \beta)$ per arm, pick top 3	Laplace-smoothed priors; 2 exploit + 1 explore per batch
MMR	$\text{argmax}\ [\lambda \cdot \text{rel}(d) - (1-\lambda) \cdot \max \text{sim}(d, \text{selected})]$	λ = 0.7, style-arm quota; greedy and exhaustive variants both tested
Confidence	P(rank 1) via 200 Dirichlet draws around w	seed 42; bands high / medium / low
Wilson interval	$(\hat{p} + z^2/2n \pm z\sqrt{(\hat{p}(1-\hat{p})/n + z^2/4n^2)}) / (1 + z^2/n)$	95% by default; used for selection rate and approval rate
Tone classifier	softmax($Wx + b$), trained at startup	800 labelled rows, weights cached to `.npz` ; probability feeds the tone feature
Deduplication	n-gram Jaccard > 0.8 → drop	Opening-line and full-text variants; seeded determinism
Analytics tests	χ^2 independence, Shannon entropy, MRR, top-1 agreement	Computed per user over their own history, with small-n caveats shown
Occasion gates	token-coverage relevance ≥ 0.5, or festive-intent bridge; plus date eligibility	Pure and clock-explicit: the caller passes today's date in the user's timezone

Slide 14 — Q&A preparation

Anticipated questions, with the suggested answer and who should take it.

Question	Who	Suggested answer
Which model writes the candidates?	B	"Groq serving `openai/gpt-oss-120b` in production, behind an adapter that also supports Gemini, OpenAI, Anthropic and Ollama. Each provider has its own circuit breaker and model, so one provider's outage or quota doesn't take the system down."
What if the model is down or out of quota?	B	"The request fails visibly: status FAILED, the reason stored, the previous batch untouched. The offline template engine is still there as a last resort, but only when the deployment opts in with `MM_ALLOW_TEMPLATE_FALLBACK=true` — and when it is used, the review screen labels the batch as template-generated with an amber banner. We removed the silent substitution that used to hide failures."
Why not just use ChatGPT?	B	"A single model call gives you one answer with no ranking and no memory. We over-generate six from one call, then filter, embed, score and diversify with our own engine — and every user action feeds back into next month's ranking. The model is one component, not the product."
How is this different from a prompt?	B	"A prompt has no state. Ours has a preference vector, Beta statistics per style arm, learned length stats and a weight vector that updates with a constrained gradient step — all visible in Settings."
Is it really automatic?	A	"Yes — APScheduler runs a daily scan, creates the cycle for the upcoming month, gathers occasion context, generates, and notifies at T-3 and T-1 days. Preparation is idempotent, so a restart or a duplicate cron tick can't create two batches."
What does 'transparent' actually mean on screen?	A	"Three things: the provider and model that wrote the batch, the six-feature score breakdown per candidate, and plain-English reasons with a link to the nearest past post it was compared against. Plus provenance on the request itself — model, source, timestamp, fallback reason."
Why SQLite and not PostgreSQL?	A	"SQLite is the zero-setup default for development and for the free hosting tier. The schema is portable — string enums, UTC-normalised timestamps, vectors as float32 blobs — so pointing `MM_DATABASE_URL` at managed Postgres is the only change. That's the next deployment step."
Isn't 22% selection rate low?	B	"n is nine, so the Wilson interval is six to fifty-five percent — too early to read as a quality signal. It's also counting 'left unselected' events. The metric exists precisely so we don't over-claim from small numbers."
How do you stop it repeating itself?	B	"Three ways: n-gram Jaccard deduplication at generation, cosine similarity against your full history with a novelty feature in the score, and MMR with style-arm quotas so the three shown are stylistically distinct. There's also a repetition alert in analytics at cosine 0.85."
Why does a calendar occasion sometimes not appear?	A	"By design. An occasion existing in the calendar is not a reason to mention it. Two gates decide: semantic relevance between your objective and the occasion

		name — plus a narrow festive-intent bridge — and date eligibility. An unrelated post never gets a holiday bolted on."
What about privacy / API keys in the browser?	A	"The frontend only calls the public API and holds no secrets. Refresh tokens are httpOnly cookies; passwords are Argon2id. Provider keys live in platform environment variables, never in the repository or the browser."
How do you know what's deployed?	A	"The health endpoint reports the git commit it is running, taken from Render's environment, alongside provider, model, embedder and database status. That turned deployment verification from guesswork into one request."
What's the hardest part you'd redo?	Either	"Getting the golden tests written first. Several real bugs — candidates never being embedded, a naive/aware datetime mix, MMR returning two candidates from the same style arm, a silently substituted template batch — were caught before they could mislead anyone."
Can it handle multiple users?	A	"Yes — every query is scoped to the authenticated user, there's per-user rate limiting, and the API is stateless so instances can be added behind a load balancer. The scheduler worker is separate and shares only the database."

Timing summary

Slide	Topic	Speaker	Cumulative
1	Title	A then B	0:30
2	The problem	A	1:30
3	The goal	B	2:15
4	User flow	A	3:15
5	Architecture (deployed)	B	4:30
6	AI pipeline (14 stages)	A	6:00
7	The maths	B	7:30
8	**Demo — review screen**	A	9:30
9	**Demo — learning loop**	B	10:45
10	Analytics	A	11:45
11	Tech stack	B	12:45
12	Quality & tests	A	13:30
13	Live & next steps	B then A	14:00
14	Q&A	both	—
A	Appendix — pipeline stages	either	backup
B	Appendix — algorithm reference	either	backup

MonthlyMuse · Presentation script for two speakers · 14 slides + 2 appendices · 14 minutes · based on the November 2026 Progress Report